

✓ Lab2: Xử lý dữ liệu & Hồi quy đa biến

Tài liệu tham khảo

[1] Ribeiro, M. T., Singh, S., & Guestrin, C. (2016). "Why should I trust you?" Explaining the predictions of any classifier. Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, 1135–1144.

<https://doi.org/10.1145/2939672.2939778>

[2] Pearl, J. (2019). The seven tools of causal inference, with reflections on machine learning. Communications of the ACM, 62(3), 54–60. <https://doi.org/10.1145/3241036>

✓ I. Nghiên cứu lý thuyết & tự luận

✓ Câu 1: Garbage In, Garbage Out (GIGO)

Trong Học máy, có câu châm ngôn kinh điển: "Garbage In, Garbage Out".

- Giải thích ý nghĩa của câu nói này trong bối cảnh xây dựng mô hình dự đoán.

Câu châm ngôn này có nghĩa là đầu vào là rác thì đầu ra cũng là rác. Chất lượng và độ tin cậy của mô hình đầu ra phụ thuộc hoàn toàn vào chất lượng của dữ liệu đầu vào

- Tìm hiểu và liệt kê ít nhất 3 vấn đề thường gặp của dữ liệu thô có thể làm giảm hiệu suất của mô hình.

1. Dữ liệu ngoại lai: là những điểm dữ liệu có giá trị khác biệt một cách bất thường so với phần lớn các điểm dữ liệu còn lại trong tập hợp
2. Dữ liệu mất cân bằng: Vấn đề này thường gặp trong các bài toán phân loại, khi số lượng mẫu của một lớp áp đảo hoàn toàn lớp còn lại
3. Dữ liệu nhiễu và không nhất quán: Đây là những dữ liệu chứa các lỗi ngẫu nhiên, sai số đo lường, lỗi đánh máy hoặc định dạng không đồng nhất

- Trích dẫn ít nhất 01 bài báo hoặc case study thực tế minh họa hậu quả của việc sử dụng dữ liệu kém chất lượng

Ribeiro, M. T., Singh, S., & Guestrin, C. (2016). "Why should I trust you?" Explaining the predictions of any classifier. Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, 1135–1144.

<https://doi.org/10.1145/2939672.2939778>

- Đưa ra các biện pháp hoặc kỹ thuật phổ biến để cải thiện chất lượng dữ liệu trước khi huấn luyện mô hình.

Làm sạch dữ liệu (Data Cleaning): Sàng lọc và loại bỏ các dữ liệu bị hỏng, nhiễu, mờ, hoặc trùng lặp (ví dụ: lọc bỏ các bức ảnh tải về từ nhiều nguồn khác nhau nhưng bị trùng lặp nội dung).

Gán nhãn nhất quán (Accurate Labeling): Đảm bảo tiêu chuẩn phân loại rõ ràng và không chồng chéo. Chẳng hạn, ranh giới giữa nhãn "tươi" (fresh) và "thối rữa" (rotten) cần được xác định bằng các đặc điểm cụ thể, tránh gây nhầm lẫn cho thuật toán.

Tăng cường dữ liệu (Data Augmentation): Áp dụng các phép biến đổi như xoay, lật, phóng to, thu nhỏ, hoặc thay đổi độ sáng, độ tương phản của hình ảnh. Điều này giúp mô hình nhận diện tốt chủ thể (như quả lựu) thay vì học vẹt góc chụp hay phong nền.

Cân bằng dữ liệu (Handling Data Imbalance): Xử lý tình trạng chênh lệch số lượng mẫu giữa các lớp (ví dụ: số ảnh quả hỏng ít hơn hẳn so với quả tươi) bằng cách lấy mẫu tăng cường (oversampling) hoặc giảm thiểu (undersampling) để mô hình không bị thiên vị.

Chuẩn hóa dữ liệu (Normalization/Standardization): Đưa toàn bộ dữ liệu về cùng một thang đo hoặc định dạng chuẩn (như đồng nhất kích thước ảnh và chuẩn hóa giá trị pixel về khoảng 0-1) để thuật toán tính toán và hội tụ nhanh hơn.

- Tại sao việc hiểu rõ dữ liệu lại quan trọng hơn việc chỉ tập trung vào thuật toán Học máy?

Thuật toán Học máy thực chất chỉ là công cụ tính toán để nhận diện các quy luật, nên nếu dữ liệu đầu vào chứa nhiều nhiễu—chẳng hạn như khi bạn xây dựng tập ảnh phân loại lựu tươi và thối rữa—mô hình sẽ dễ dàng học vẹt các tương quan giả và thất bại hoàn toàn trong thực tế.

✓ Câu 2: Tương quan (correlation) và nhân quả (causation)

- Hãy giải thích sự khác biệt giữa "Tương quan" và "Nhân quả"

1. Tương quan (Correlation) Tương quan xảy ra khi hai yếu tố (hay biến số) có xu hướng thay đổi cùng nhau. Khi yếu tố A thay đổi, yếu tố B cũng có xu hướng thay đổi theo (có thể cùng tăng, hoặc một cái tăng một cái giảm). Tuy nhiên, A không tạo ra B, và B cũng không tạo ra A.
 2. Nhân quả (Causation) Nhân quả xảy ra khi sự thay đổi của một yếu tố (nguyên nhân) trực tiếp tạo ra sự thay đổi ở yếu tố kia (kết quả). Tức là A trực tiếp sinh ra B.
- Cho ví dụ minh họa một trường hợp hai biến có tương quan cao nhưng không có quan hệ nhân quả (spurious correlation).

Ví dụ: Chiều cao của trẻ em và Vốn từ vựng

Tương quan (Dữ liệu): Nếu bạn kiểm tra ngẫu nhiên học sinh từ lớp 1 đến lớp 12, bạn sẽ thấy một biểu đồ tuyệt đẹp: Trẻ em có chiều cao càng lớn thì làm bài kiểm tra từ vựng càng được điểm cao.

Suy luận sai (Nhân quả): Kéo dài chân cho trẻ sẽ giúp trẻ thông minh hơn và biết nhiều chữ hơn.

- Tại sao việc nhầm lẫn giữa tương quan và nhân quả có thể dẫn đến những quyết định sai lầm trong thực tế?

Sai lệch cốt lõi: Việc nhầm lẫn khiến chúng ta lãng phí nguồn lực để can thiệp vào "phần ngọn" (biểu hiện) thay vì giải quyết triệt để "phần gốc" (nguyên nhân thực sự).

Hậu quả thực tiễn: Các quyết định và hành động dựa trên tương quan giả sẽ hoàn toàn vô tác dụng, thậm chí gây thiệt hại nặng nề.

Rủi ro công nghệ: Trong học máy, nếu một mô hình phân loại tình trạng quả lựu tươi hay thối rữa chỉ học vẹt các tương quan ngẫu nhiên, thuật toán sẽ dự đoán sai hoàn toàn khi áp dụng vào môi trường thực tế.

- Làm thế nào để kiểm tra xem một mối quan hệ có phải là nhân quả hay không trong bối cảnh Học máy?

Trong học máy, cách tiêu chuẩn để xác định nhân quả là thực hiện các thử nghiệm chủ động như Thử nghiệm ngẫu nhiên có đối chứng (RCT) hoặc A/B testing, nơi bạn cố tình can thiệp vào một biến duy nhất để đo lường tác động trực tiếp của nó. Nếu chỉ có dữ liệu quan sát lịch sử, bạn cần áp dụng các kỹ thuật Suy luận nhân quả (Causal Inference) thông qua đồ thị nhân quả (Causal Graphs) để toán học hóa và bóc tách các biến nhiễu. Bạn có muốn tôi giải thích chi tiết hơn về cách thiết lập một đồ thị nhân quả để ngăn thuật toán học vẹt các tương quan giả không?

- Trích dẫn ít nhất 01 nghiên cứu hoặc bài báo khoa học liên quan đến chủ đề này

Pearl, J. (2019). The seven tools of causal inference, with reflections on machine learning. Communications of the ACM, 62(3), 54–60. <https://doi.org/10.1145/3241036>

✓ II. Kỹ thuật tiền xử lý dữ liệu

Bước 1: Tạo dữ liệu giả lập (Dummy Data) có cá nhân hóa

```
import pandas as pd
import numpy as np

k = 3 % 10 #k = ( Số cuối MSSV % 10)

# Tạo dictionary chứa dữ liệu mẫu
```

```
data = {
    'QuocTich ': ['France ', 'Spain ', 'Germany ', 'Spain ', 'Germany ', 'Fran
    'Tuoi ': [44 , 27 , 30 , 38 , 40 , 35 , np.nan , 48 , 50 , 37] ,
    'MucLuong ': [72000 + k *1000 , 48000 + k *1000 , 54000 , 61000 , np.nan ,
    'DaMuaHang ': ['No ', 'Yes ', 'No ', 'No ', 'Yes ', 'Yes ', 'No ', 'Yes ',
}

df = pd.DataFrame(data) # Dùng pandas để biến data thành kiểu dữ liệu DataFr
print("Dữ liệu gốc của MSSV đầu", k , ":\n", df )
```

Dữ liệu gốc của MSSV đầu 3 :

	QuocTich	Tuoi	MucLuong	DaMuaHang
0	France	44.0	75000.0	No
1	Spain	27.0	51000.0	Yes
2	Germany	30.0	54000.0	No
3	Spain	38.0	61000.0	No
4	Germany	40.0	NaN	Yes
5	France	35.0	58000.0	Yes
6	Spain	NaN	52000.0	No
7	France	48.0	82000.0	Yes
8	Germany	50.0	83000.0	No
9	France	37.0	67000.0	Yes

Bước 2: Xử lý dữ liệu bị thiếu (Missing Values)

```
from sklearn.impute import SimpleImputer
```

```
# Khởi tạo SimpleImputer, chỉ định giá trị khiếm khuyết là np.nan và chiến l
imputer = SimpleImputer(missing_values = np.nan, strategy = 'mean')
# fit(): tính toán và ghi nhớ giá trị trung bình của riêng 2 cột này
imputer.fit(df.iloc[:, 1:3])
# transform() sử dụng giá trị trung bình đã học được để lấp vào các ô NaN
df.iloc[:, 1:3] = imputer.transform(df.iloc[:, 1:3])
```

```
print(df)
```

	QuocTich	Tuoi	MucLuong	DaMuaHang
0	France	44.000000	75000.000000	No
1	Spain	27.000000	51000.000000	Yes
2	Germany	30.000000	54000.000000	No
3	Spain	38.000000	61000.000000	No
4	Germany	40.000000	64777.777778	Yes
5	France	35.000000	58000.000000	Yes
6	Spain	38.777778	52000.000000	No
7	France	48.000000	82000.000000	Yes
8	Germany	50.000000	83000.000000	No
9	France	37.000000	67000.000000	Yes

Bước 3: Mã hóa dữ liệu phân loại

```
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import OneHotEncoder, LabelEncoder
```

```
# Tách dữ liệu thành Ma trận đặc trưng X (chứa các cột đầu vào) và Vector mục
X = df.iloc[:, :-1].values
y = df.iloc[:, -1].values

# Label encoding: Áp dụng cho cột DaMuaHang (Biến mục tiêu: No=0, Yes=1).
le = LabelEncoder()
y= le.fit_transform(y)

# One-hot encoding: Áp dụng cho cột QuocTich
ct = ColumnTransformer(
    transformers=[('encoder', OneHotEncoder(), [0])],
    remainder='passthrough'
)

X = np.array(ct.fit_transform(X))
```

```
print(X)
```

```
[[1.0 0.0 0.0 44.0 75000.0]
 [0.0 0.0 1.0 27.0 51000.0]
 [0.0 1.0 0.0 30.0 54000.0]
 [0.0 0.0 1.0 38.0 61000.0]
 [0.0 1.0 0.0 40.0 64777.77777777778]
 [1.0 0.0 0.0 35.0 58000.0]
 [0.0 0.0 1.0 38.77777777777778 52000.0]
 [1.0 0.0 0.0 48.0 82000.0]
 [0.0 1.0 0.0 50.0 83000.0]
 [1.0 0.0 0.0 37.0 67000.0]]
```

```
print(y)
```

```
[0 1 0 0 1 1 0 1 0 1]
```

Bước 4: Chia tập dữ liệu (Splitting)

```
from sklearn.model_selection import train_test_split
X_train, X_test, Y_train, Y_test = train_test_split(X, y, test_size = 0.2, r
```

```
print(X_train)
print()
print(X_test)
```

```
[[0.0 0.0 1.0 27.0 51000.0]
 [0.0 1.0 0.0 30.0 54000.0]
 [1.0 0.0 0.0 37.0 67000.0]
 [0.0 0.0 1.0 38.77777777777778 52000.0]
 [1.0 0.0 0.0 48.0 82000.0]
 [1.0 0.0 0.0 44.0 75000.0]
 [0.0 0.0 1.0 38.0 61000.0]
 [0.0 1.0 0.0 50.0 83000.0]]

[[1.0 0.0 0.0 35.0 58000.0]
 [0.0 1.0 0.0 40.0 64777.77777777778]]
```

```
print(Y_train)
print()
print(Y_test)
```

```
[1 0 1 0 1 0 0 0]
```

```
[1 1]
```

Bước 5: Chuẩn hóa dữ liệu (feature scaling)

```
from sklearn.preprocessing import StandardScaler

sc = StandardScaler()

# Chuẩn hóa tập huấn luyện
# Lấy cột Tuổi (index 3) và Lương (index 4) ra để xử lý.
# Hàm fit_transform sẽ vừa tính toán trung bình/độ lệch chuẩn, vừa thay đổi
# Sau đó, gán đè kết quả đã chuẩn hóa lại vào đúng vị trí của 2 cột đó trong
X_train[:, 3:] = sc.fit_transform(X_train[:, 3:])

# chuẩn hóa tập thử nghiệm
# transform() - sử dụng lại các tham số đã học từ X_train
X_test[:, 3:] = sc.transform(X_test[:, 3:])
```

```
print(X_train)
print()
print(X_test)
```

```
[[0.0 0.0 1.0 -1.600608128076617 -1.1863051173273855]
 [0.0 1.0 0.0 -1.2036720136511871 -0.9429604778756141]
 [1.0 0.0 0.0 -0.2774877466585179 0.11153295974872855]
 [0.0 0.0 1.0 -0.0422663455175224 -1.1051902375101283]
 [1.0 0.0 0.0 1.177944672901391 1.3282561570075855]
 [1.0 0.0 0.0 0.6486965203341514 0.7604519982867856]
 [0.0 0.0 1.0 -0.14517570851670797 -0.37515631915481423]
 [0.0 1.0 0.0 1.4425687491850108 1.4093710368248427]]

[[1.0 0.0 0.0 -0.5421118229421377 -0.6185009586065856]
 [0.0 1.0 0.0 0.11944836776691183 -0.06872232873406481]]
```

✓ III. Hồi quy tuyến tính đa biến

Bước 1: Tải và Khám phá dữ liệu

```
from sklearn . datasets import fetch_california_housing

# Khởi tạo bộ dữ liệu vào biến housing
housing = fetch_california_housing()

# Chuyển sang Pandas
```

```
df_housing = pd.DataFrame(housing.data, columns = housing.feature_names)
df_housing['Target_Price'] = housing.target
```

```
# Hiển thị 5 dòng đầu của dữ liệu
print(df_housing.head())
```

```

      MedInc  HouseAge  AveRooms  AveBedrms  Population  AveOccup  Latitude  \
0   8.3252    41.0    6.984127   1.023810      322.0    2.555556    37.88
1   8.3014    21.0    6.238137   0.971880     2401.0    2.109842    37.86
2   7.2574    52.0    8.288136   1.073446      496.0    2.802260    37.85
3   5.6431    52.0    5.817352   1.073059      558.0    2.547945    37.85
4   3.8462    52.0    6.281853   1.081081      565.0    2.181467    37.85

      Longitude  Target_Price
0    -122.23      4.526
1    -122.22      3.585
2    -122.24      3.521
3    -122.25      3.413
4    -122.25      3.422

```

Câu hỏi 1: Dữ liệu này có bao nhiêu đặc trưng (features)? Kích thước của tập dữ liệu là bao nhiêu?

Bảng 1: Thông tin bộ dữ liệu California Housing

Số mẫu	20640
Số đặc trưng	8
Kiểu dữ liệu	Số liên tục
Mục tiêu	Giá nhà [0,15, 5,0] (đơn vị 100,000 USD)

Dữ liệu này có 8 đặc trưng

Tập này có 20640 hàng x 9 cột.

Câu hỏi 2: Có biến nào bị thiếu (NaN) không?

Không có giá trị nào bị thiếu trong tập dữ liệu

```
print(df_housing.isnull().sum())
```

```

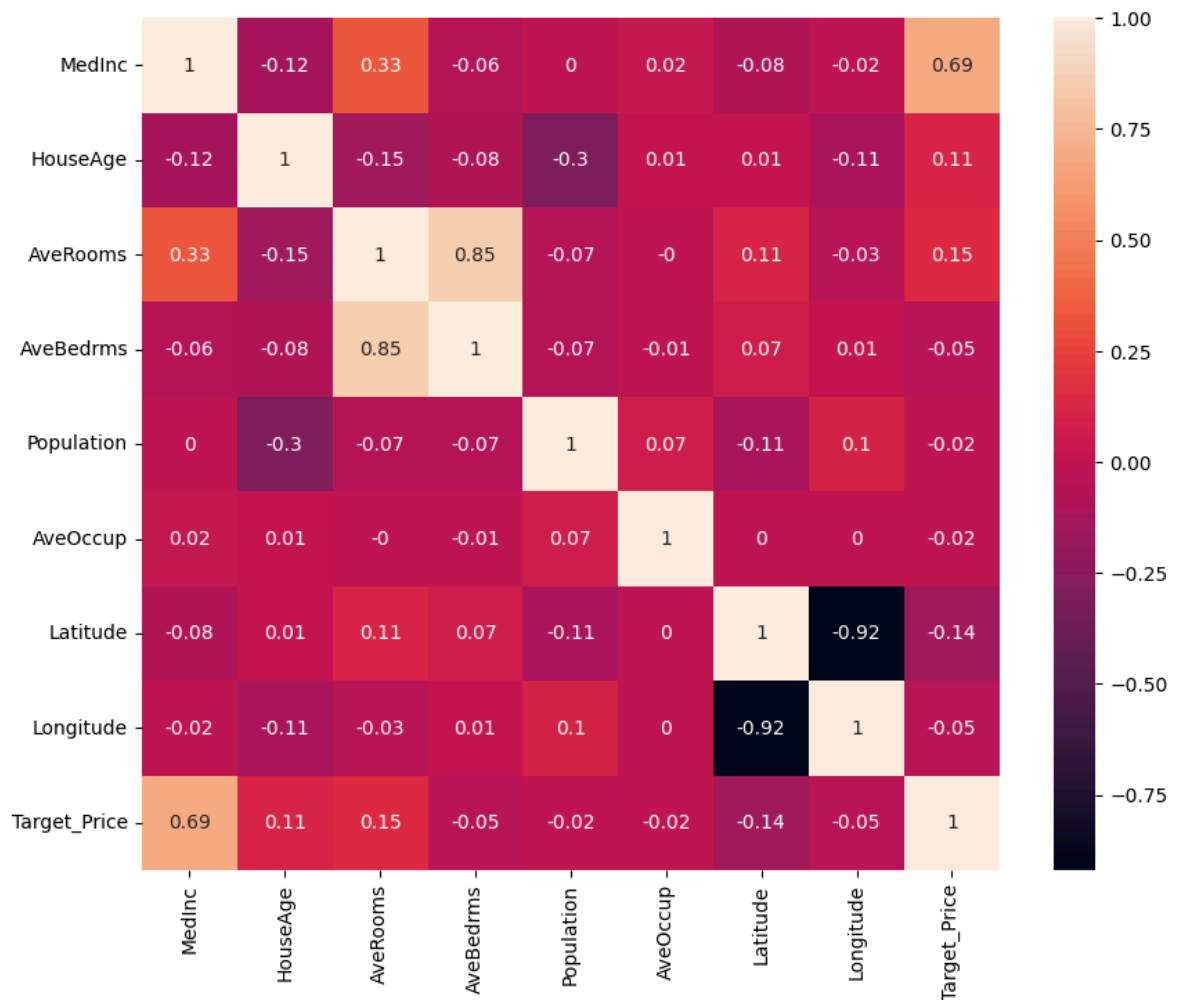
MedInc      0
HouseAge    0
AveRooms    0
AveBedrms   0
Population  0
AveOccup    0
Latitude     0
Longitude    0
Target_Price 0
dtype: int64

```

Bước 2: Phân tích tương quan (correlation matrix)

```
import seaborn as sns
import matplotlib.pyplot as plt

plt.figure(figsize=(10,8))
correlation_matrix = df_housing.corr().round(2)
# Code vẽ heatmap
sns.heatmap(data = correlation_matrix, annot = True)
plt.show ()
```



Nhận xét: Dựa trên output được in ra, biến nào có tương quan dương cao nhất với giá nhà? Biến nào có tương quan âm cao nhất? Làm sao để biết biến nào ảnh hưởng mạnh nhất? (Sinh viên tự trả lời vào block Markdown của Notebook).

- Biến Med_Inc có tương quan dương cao nhất với giá trị là 0.69

- Biến Latitude có tương quan âm cao nhất với giá trị -0.14
- Để xác định mức độ ảnh hưởng (cường độ) của một biến lên biến mục tiêu, chúng ta phải xét giá trị tuyệt đối của hệ số tương quan, ký hiệu toán học là $|r|$.

Bước 3: Xây dựng mô hình Multiple Linear Regression

```
# Chọn đặc trưng và mục tiêu
X_housing = df_housing.iloc[:, :-1].values
y_housing = df_housing.iloc[:, -1].values

# Chia thành tập huấn luyện và tập kiểm thử
X_train, X_test, Y_train, Y_test = train_test_split(X_housing, y_housing, te

# Huấn luyện mô hình
from sklearn.linear_model import LinearRegression # import thư viện LinearRe

# Tạo một đối tượng (object) mang tên lr_housing từ lớp LinearRegression.
lr_housing = LinearRegression()

# fit(): thuật toán Bình phương tối thiểu sẽ chạy ngầm để tìm ra
# các hệ số góc (weights) và hệ số tự do (bias) sao cho tổng bình phương
# sai số giữa giá nhà dự đoán và giá nhà thực tế trong tập Y_train là nhỏ nh
lr_housing.fit(X_train, Y_train)
```

▼ LinearRegression ⓘ ?

LinearRegression()

Bước 4: Đánh giá mô hình

```
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score

# Dự đoán trên tập kiểm thử
y_pred = lr_housing.predict(X_test)

# 1. Tính MAE (Mean Absolute Error)
mae = mean_absolute_error(Y_test, y_pred)
print(f'MAE (Sai số tuyệt đối trung bình): {mae:.4f}')
```

2. Tính RMSE (Root Mean Squared Error)

Đầu tiên tính MSE (bình phương các sai số rồi lấy trung bình) bằng mean_sq

Sau đó dùng np.sqrt() để lấy căn bậc hai, trả đơn vị về cùng hệ quy chiếu

```
mse = mean_squared_error(Y_test, y_pred)
rmse = np.sqrt(mse)
print(f'RMSE (Căn bậc hai sai số bình phương trung bình): {rmse:.4f}')
```

3. Tính R² Score (Coefficient of Determination)

```
r2 = r2_score(Y_test, y_pred)
print(f'R2 Score (Hệ số xác định): {r2:.4f}')
```

MAE (Sai số tuyệt đối trung bình): 0.5278
RMSE (Căn bậc hai sai số bình phương trung bình): 0.7336
R2 Score (Hệ số xác định): 0.5931